

★ WEEK 8 · PART 1 · PRACTICE

State Space, Worked by Hand

Extract a Fair Value from Noise; Separate Level from Trend

The two equations, made concrete on real financial problems

Part 1's ideas, with the arithmetic shown. We write two financial problems in state-space form, then compute - a fair value pulled from a noisy print, and an index's level and momentum separated by hand.

1P

PART 1 · APPLICATIONS

Two Hidden States, Computed by Hand

Case A: the local level model pulls a fair value out of noisy prints in one filtering step. Case B: the local linear trend model splits an index into a level and a slope. Then a Claude Code lab.

What Part 1 Told Us

Now we make it concrete - with numbers

1

Two equations describe everything

An observation equation for what you see, and a state equation for the hidden thing that drives it.

2

The local level model

A hidden value drifts; you observe it through noise. The model separates the signal from the noise - we do one step by hand.

3

The local linear trend model

Add a slope, and the level can drift with momentum. We predict and update both, by hand, on a real index.

Everything here is a calculator exercise: two variances, one noisy print, one index surprise.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Write the problem in state-space form.
- Compute one step of the model by hand.
- Then read what the hidden state tells you.

What you should watch

- How much to trust a single noisy print.
- How a surprise updates a level and a slope.
- How one template fits many finance problems.

A Fair Value Hidden in Noisy Prints

The local level model, made concrete

A stock's true fair value drifts slowly, but every print you see is contaminated by noise - bid-ask bounce, thin liquidity, a stray trade. The local level model treats fair value as a hidden state and the print as a noisy observation. We compute the fair value from one print by hand

The two pieces

- Hidden state: the true fair value, drifting.
- Observation: the noisy printed price.
- Noise is larger than the drift here.

What we'll do by hand

- Write the observation and state equations.
- Compute the signal-to-noise ratio and gain.
- Turn a 100.60 print into a fair value.

Two Equations for Fair Value

What you see, and what is hidden

$$\begin{aligned} \text{observation: } P_t &= L_t + \varepsilon_t, & \varepsilon_t &\sim N(0, 0.25) \text{ (noisy print)} \\ \text{state: } L_t &= L_{t-1} + \eta_t, & \eta_t &\sim N(0, 0.04) \text{ (true value drifts)} \end{aligned}$$

The observation

- The print equals fair value plus noise.
- Noise variance is 0.25 - prices are noisy.
- You never see the fair value directly.

The state

- Fair value takes a small random step each day.
- Drift variance is only 0.04 - it moves slowly.
- This is the hidden thing we want.

Signal vs Noise

How much of a print is real?

Quantity	Value	Meaning
state (drift) variance	0.04	how fast fair value moves
obs (noise) variance	0.25	how noisy each print is
signal-to-noise q	0.16	drift / noise - noise dominates

A low q means most of a print's move is noise, not real - so a new print should move our estimate only a little.

One Filtering Step

From a 100.60 print to a fair value

$$\text{predict: } R = C + \sigma_{\eta}^2 = 0.10 + 0.04 = 0.14$$

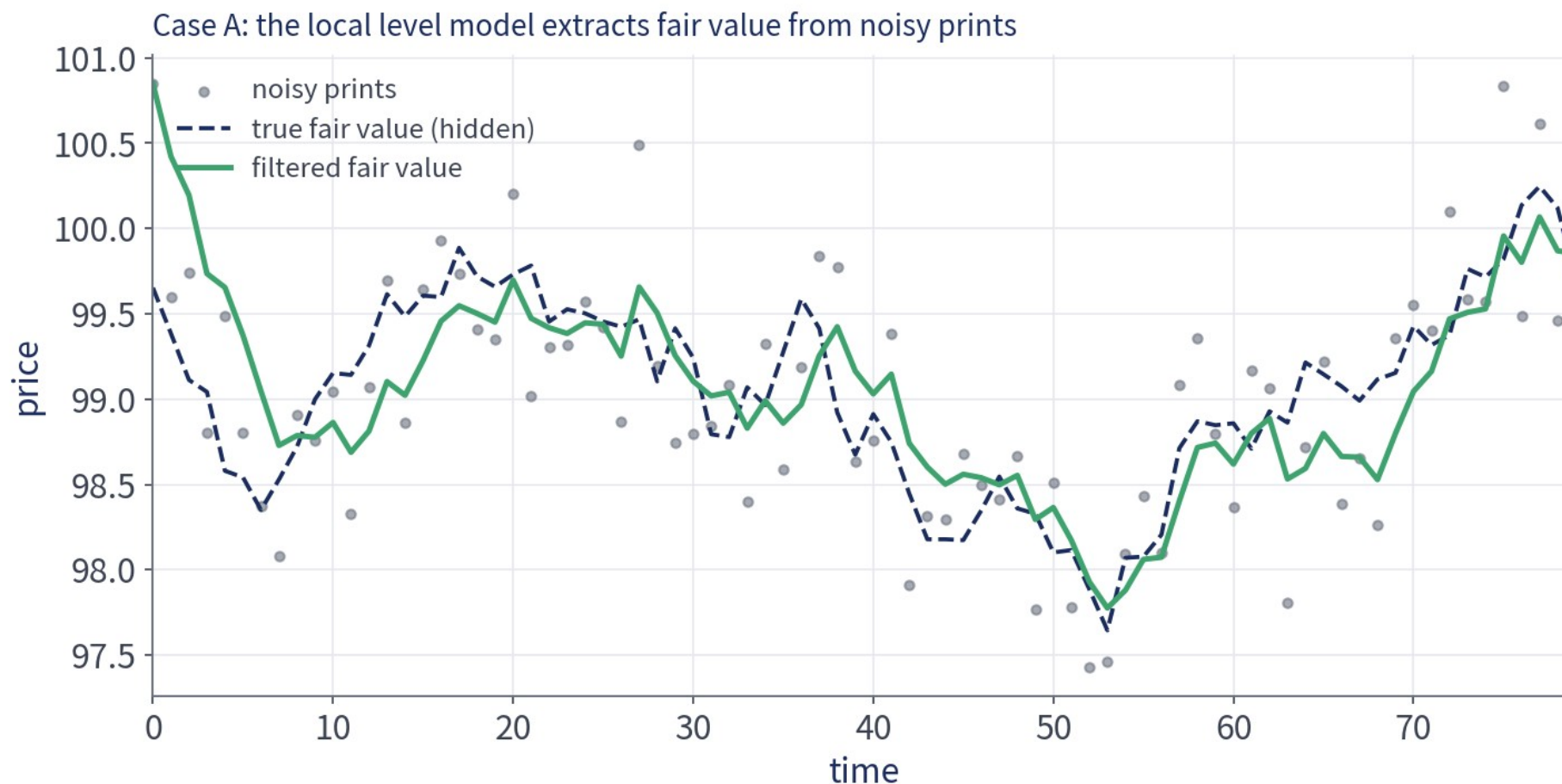
$$\text{gain: } K = \frac{R}{R + \sigma_{\varepsilon}^2} = \frac{0.14}{0.14 + 0.25} = 0.36$$

$$\text{fair value: } 100.00 + 0.36 (100.60 - 100.00) = \mathbf{100.22}$$

The gain is only 0.36, so the 100.60 print pulls our estimate just to 100.22 - we trust the noisy print about a third of the way.

Fair Value, Pulled from Noise

Run that one step over a whole series and the filtered fair value (green) tracks the hidden true value (dashed) while ignoring most of the noise in the scattered prints. The gain decides how closely it follows each print



What the Gain Buys You

A disciplined way to trust a price

Without the model

- You either chase every noisy tick,
- or you pick an arbitrary moving average.
- Neither knows how noisy the data really is.

With the model

- The gain comes from the signal-to-noise ratio.
- Noisier prints are trusted less, automatically.
- You get a fair value, not a guess.

An Index's Level and Its Momentum

The local linear trend model, made concrete

Sometimes the hidden state is moving. An index has a level and a slope - where it is, and how fast it's climbing. The local linear trend model tracks both, and lets the slope itself drift. We predict the next level and update both level and slope from one surprise, by hand

The two hidden pieces

- Level: where the index is now.
- Slope: how fast it's moving (momentum).
- Both are hidden; both can change.

What we'll do by hand

- Predict the next level from level plus slope.
- Measure the surprise against a new print.
- Nudge the level and the slope.

Two Equations for Level and Trend

The slope drifts too

$$\begin{aligned} \text{level: } L_t &= L_{t-1} + S_{t-1} + \text{noise} \\ \text{slope: } S_t &= S_{t-1} + \text{noise} \quad (\text{the drift itself drifts}) \end{aligned}$$

The level

- Next level = level + slope + noise.
- The slope carries it forward each step.
- So momentum is built in.

The slope

- Next slope = slope + noise.
- The drift itself can slowly change.
- That is what 'linear trend' means here.

Predict, Then Update

One surprise moves both level and slope

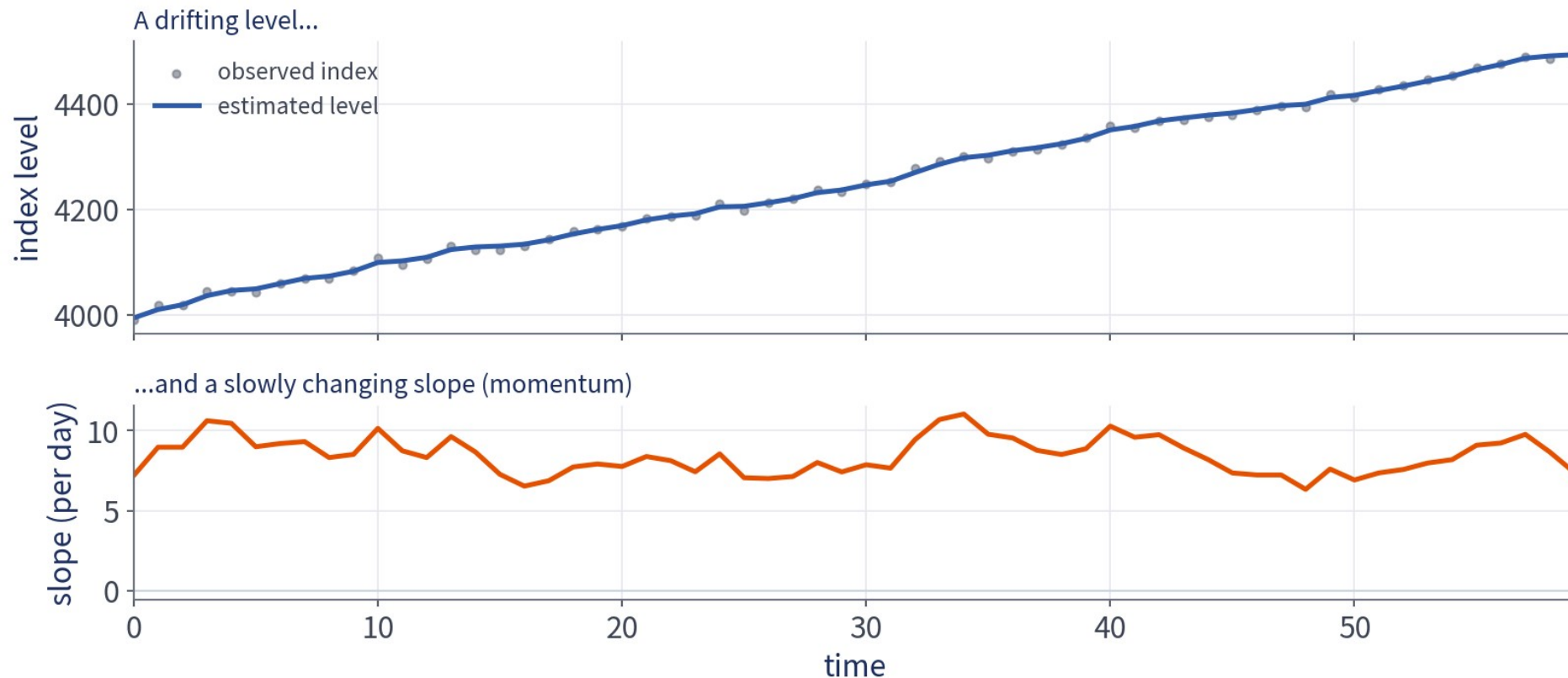
$$\begin{aligned}\text{predict next level} &= L + S = 4000 + 5 = 4005 \\ \text{new obs } 4012 &\Rightarrow \text{surprise} = 4012 - 4005 = +7 \\ \text{level} &\leftarrow 4005 + 0.5(7) = \mathbf{4008.5}, \quad \text{slope} \leftarrow 5 + 0.1(7) = \mathbf{5.7}\end{aligned}$$

The +7 surprise raises the level to 4008.5 and nudges the slope up to 5.7 - the model now believes the index is climbing a little faster.

Level and Slope, Separated

Repeated over a series, the model tracks a smooth level through the noisy index (top) while separately estimating the slope - the momentum - underneath (bottom). Two hidden states, read off one noisy observation

Case B: local linear trend separates the level from the drift



Why Separate Level and Slope

Momentum you can actually measure

What it gives you

- A clean level, free of day-to-day noise.
- A slope estimate - the momentum - as a number.
- Both updated the moment new data arrives.

Why it matters

- A rising slope is a momentum signal.
- A flattening slope warns a trend is fading.
- You separate 'where' from 'how fast'.

The Common Pattern

One template, two hidden states

Both cases follow the same recipe. Decide what you observe and what is hidden, write the two equations, and let the model combine your prior belief with each noisy observation. Fair value or momentum, the machinery is identical - and it is the whole point of state space

The recipe

- Name the observation and the hidden state.
- Write the observation and state equations.
- Combine prior and data by the gain.

Why it's powerful

- The same code fits many finance problems.
- The gain adapts to how noisy the data is.
- Part 2 turns one step into a full filter.

State Space in Finance

The same two-equation template fits problem after problem: fair value from noisy prints, a drifting trend, a time-varying beta, yield-curve factors, stochastic volatility, and nowcasting. Name what's hidden and the filter does the rest

State Space in Finance: What's Hidden?

The same two-equation template - a hidden state, a noisy observation - fits problem after problem.

Fair value

observe: noisy prints / quotes

hidden state:

the true price behind the noise

Trend & momentum

observe: observed index

hidden state:

a drifting level and slope

Time-varying beta

observe: stock vs market returns

hidden state:

a hedge ratio that moves

Term structure

observe: observed bond yields

hidden state:

a few hidden yield factors

Stochastic vol

observe: observed returns

hidden state:

a hidden volatility level

Nowcasting

observe: mixed-frequency data

hidden state:

a hidden state of the economy

One template, many problems

Write down what you observe and what is hidden; the Kalman filter (Part 2) then does the rest.

Reproduce the Hand Calcs, Then Scale Up

Direct Claude Code to check your arithmetic

GOAL Verify both models in code, then run them over a real series.

[INPUT] Prompt · Claude Code

Write and run `part1_practice.py` that:

CASE A - local level:

- * set state var 0.04, obs var 0.25; print q ;
- * from prior 100.00 (var 0.10) and print 100.60, compute the gain and the filtered fair value;
- * then filter a real noisy price series.

CASE B - local linear trend:

- * from level 4000, slope 5, and obs 4012, predict and update level and slope;
- * then track level and slope on a real index.

Match every printed number to the slides.

Check against the slides

- signal-to-noise $q = 0.16$.
- gain $K = 0.36$.
- fair value = 100.22.
- level 4008.5, slope 5.7.

Deliverable: `part1_practice.py` whose printed numbers match the hand calculations, then the charts.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 What decides how much a print moves your estimate?

The gain, from the signal-to-noise ratio

Q2 In the trend model, what does the slope represent?

The momentum - how fast the level moves

Q3 What do the two equations describe?

What you observe, and the hidden state

All three come from the local level filter, the local linear trend update, and the state-space template.

Three Pitfalls to Avoid

Where state-space setups slip

Common mistakes

- "A noisy print is the fair value."
- "The level and the trend are the same thing."
- "Bigger gain is always better."

The reality

- The print is the state plus noise - filter it.
- Level is where; slope is how fast - keep them apart.
- The right gain comes from the signal-to-noise ratio.

Two Calculations to Remember

Part 1 applications in one screen

1

Pull a fair value from a noisy print

With state variance 0.04 and noise variance 0.25, the signal-to-noise ratio is 0.16 and the gain is 0.36 - so a 100.60 print moves the fair value only to 100.22. The model trusts noisy data less, automatically.

2

Separate an index's level from its slope

Predicting level + slope gave 4005; a +7 surprise raised the level to 4008.5 and the slope to 5.7. The local linear trend model tracks where the index is and how fast it's moving, as two numbers.

3

One template, many problems

Name what you observe and what is hidden, write two equations, and combine prior and data by the gain - the same recipe fits fair value, momentum, beta, yields, and volatility.

Next — Part 2 practice: the full Kalman filter and smoother, run over a series by hand and in code.